

Python Tip: `try...except`

Does one unexpected bad value cause your entire script to crash?

Use `try...except` to skip the bad value and keep going!



One Bad Value Stops Everything

Imagine we read a list of prices from a file. Each price is text, so the loop uses `float()` to convert it to a number. One unexpected entry is `"missing"`.

```
prices = ["12.50", "missing", "8.25"]

for price_text in prices:
    price = float(price_text)  # This will crash!
    print("Price:", price)
```

The value `"missing"` is not a valid number, so `float()` crashes. The crash stops the loop before it reaches the last value.

Use a `try` Block

If we want to run code but protect the script when something goes wrong, we can use a `try` block. A `try` block looks like this:

```
price_text = "missing"

try:
    price = float(price_text)
    print("Price:", price)
except:
    print("Could not convert:", price_text)
```

```
Could not convert: missing
```

Python runs the code under `try` first. `float()` cannot convert `"missing"`, so the first `print()` does not run. Python then runs the `print()` under `except:` instead.

Put It Inside the Loop

We can put the `try` block inside the loop so that one bad value does not crash the script. If a price cannot be converted, `continue` moves the loop to the next value.

```
prices = ["12.50", "missing", "8.25"]

for price_text in prices:
    try:
        price = float(price_text)
    except:
        print("Skipped:", price_text)
        continue

    print("Price:", price)
```

```
Price: 12.5
Skipped: missing
Price: 8.25
```

The loop skips `"missing"` and still prints the final price.

Catch Only Certain Errors

So far, `except:` has caught every error. Python gives each type of error a name. Bad number text causes `ValueError`, while division by zero causes `ZeroDivisionError`.

We can put an error name after `except` to catch only that type. Other errors can still crash the script, which helps us **fail fast** when an unexpected bug happens.

```
try:
    price = float("missing")
except ValueError:
    print("Bad price") # This runs

try:
    result = 10 / 0 # Raises ZeroDivisionError
except ValueError:
    print("Bad price") # This does not run

# ZeroDivisionError still crashes the script!
```

`except ValueError:` catches the bad price text but not division by zero. The unrelated division error passes through instead of being hidden.

Wrap-Up

Use `try...except` when one unexpected bad value should not stop all the remaining work.

Put the code that may crash under `try`. Name the error you want to catch after `except`.

Only a matching error is caught. Other errors still crash.

Follow me for more tips.

- Shep4